

AI 编程先分 AFK 和 HITL

用 3 个问题判断，哪些任务能交给 Agent 独立推进

适合用 Agent 做产品的人

别实时盯着 Agent 写代码

把人的参与点前移到定义任务，再后移到验收结果。

边猜边写

需求仍然模糊，就让 Agent 一边猜产品决策，一边堆实现。

先建反馈闭环

人先定目标、边界和验收，Agent 执行后再由人检查结果。

AFK 不是完全不管，而是把实时监督换成事前定义和事后审查。

这 3 个问题决定能否 AFK

三个问题都能答清楚，Agent 才适合离开人的实时监督独立推进。

01 目标和范围是否明确？

Agent 知道要改什么，也知道哪些内容这次不做。

02 结果能否自动判断？

测试、类型检查、构建或 lint 能给出明确反馈。

03 失败能否复现和回收？

有验收标准、错误信息或 QA 步骤，问题能重新进入任务队列。

任意一个问题答不上来，先留在 HITL。

把开发拆成三段

HITL 和 AFK 会在一条流程里交替出现，不是二选一。

01 HITL: 澄清需求，统一领域语言，确认 PRD 和范围。

02 AFK: 按垂直切片实现，跑测试、类型检查和自动审查。

03 HITL: 手动走用户路径，把 QA 问题写成新任务。

人守住决策点，Agent 接走边界清楚的执行块。

这 3 类工作必须有人判断

缺少业务判断时，Agent 会把模糊需求直接写成代码。

-
- ☐ 产品取舍和领域边界：为什么做、做多少、哪些内容不做。
 - ☐ UI、命名和体验：文案是否自然，路径是否符合真实用户习惯。
 - ☐ 架构与高风险操作：接口怎么划分，失败后会影响什么。

这些任务需要人理解业务和后果，不能只靠执行力。

把 issue 写成 AFK 任务

内容不必很长，但要让 Agent 知道目标、边界、反馈和停止条件。

任务目标：……

为什么做：……

范围：包含……；不包含……

验收标准：

1. ……

2. ……

反馈方式：测试 / 类型检查 / 构建 / QA 步骤

停止条件：遇到……时暂停并请求确认

“停止并请求确认”能防止 Agent 在未知分支里继续猜。

AFK Loop 只跑反馈闭环

任务一旦被阻塞，或需要新增产品判断，就退出 AFK。

01 读取任务，确认没有阻塞

02 探索代码，确认模块和接口

03 先写失败测试，再补实现

04 跑完检查，记录结果和遗留问题

并行只用于依赖清楚、相互独立的垂直切片。

回来后看 4 个证据

完成不是一句“已修复”，而是可以逐项检查的结果。

- ☐ 每条验收标准都有对应结果。
- ☐ 测试、类型检查、构建和 lint 结果可见。
- ☐ 真实用户路径已经由人手动走过。
- ☐ 新问题已经写成任务，没有混进当前范围。

AI 可以生成 QA plan，体验、术语和边界仍由人判断。

第一次练手，从一个小 bug 开始

澄清 → PRD → 垂直切片 → AFK 实现 → HITL 验收

常见的 3 个坑

01 把 HITL 任务伪装成 AFK

02 按数据库、API、前端水平拆分

03 没有测试和可复现的反馈

挑一个复现步骤清楚的小 bug，用第 6 张模板写成任务，跑完后手动验收。